Date: 14 Oct, 2025

Data in RAM: volatile, disappears once the program stops running
Data Saved in file remains there even after prog stops running (or system reboots)

Output File -> A file that data is written to.

3 Main steps in FILE Handling:

1. Open the file.
2. Process the file
3. Close the file.

Types of files: Text and Binary

- A text file contains data that has been encoded as text using, schemes such as ASCII or Unicode (the data is stored as a series of characters).

- A binary file contains data that has not been converted as text.

File Access Methods:

- Sequential access: Access from beginning of file to the end of the file.
- Direct Access: Jump directly to any location without reading the data.

Filenames: Each Operating System has its own roles for naming files. Many systems support the use of filename extension.

File Object: Programs creates a file object in-memory to work with a file.

A file object is an object that is associated with a specific file and provides mechanisms to work with a file. (In program, a variable refers a file object)

—----------

Open a File:

file_var = open(filename, mode)

mode:

‘r’: reading only
‘w’: open for writing. Erase existing files
‘a’: Append data written at the end.
‘r+’: read and write (but don’t create a file if it doesn’t exist
‘w+’: read and write (create a file if it doesn’t exist)
‘rb’ : read binary

‘wb’: write binary

Writing data to a file:

file_var.write(string)

- Once a program is finished working with a file, it should close the file.
- In some systems, failure to close can cause loss of data.

This happens because the data that is written to a file is first written to a buffer, which is a small holding section in memory. This technique increases system’s performance.

file_write.py

```
outfile = open(‘temp.txt’, ‘w’)
outfile.write(‘hello\n’)
outfile.write(‘world\n’)
outfile.write(‘bye bye\n’)
outfile.close()
```

Contents of file after above write:

‘Hello\nworld\nbye bye\n’

The ‘\n’ causes each item to appear in a separate line when used in a text editor.

—------

Reading a file:

file_read.py

```
infile = open(‘temp.txt’, ‘r’)
file_contents = infile.read()
infile.close()
print(file_contents)
```

‘read’ method reads the entire content of a file with one statement.

‘readline’ method reads a line from a file (A line is simply a string of characters terminated by \n.

line_read.py

```
infile = open(‘temp.txt’, ‘r’)
line1 = infile.readline()
line2 = infile.readline()
line3 = infile.readline()

infile.close()
print(line1)
print(line2)
print(line3)
```

Read/Write Position: When a file is opened for reading, a special value known as ‘read position’ is internally maintained for that file. It marks the location of next item to read/written. ‘readline’ in a loop

```
f = open(‘temp.txt’, ‘r’)
l = f.readline()
while (l != ‘’):
        print(l)
        l = f.readline()
f.close()
```

—---

Exercise: WAP to enter 2 names and write them to a file. (Note: When a program writes data that has been entered by the user to a file, it is necessary to concatenate a \n. This ensures that each piece of data is on a separate line.

Exercise: WAP to read the content of two files and merge them into one file.

—

Seek and Tell

File: tmp.txt: ‘abcdefghijkl’

```
f = open(‘tmp.txt’, ‘r’)
f.read()
f.tell()
f.seek(0)
f.tell()
f.read(4)
f.tell()
f.write(‘123’)
f.tell()
f.seek(0)
f.read()
```

—-------

readlines() and writelines()

```
f = open(‘tmp’, ‘w’)
data = [‘abc\n’, ‘pqr\n’, ‘xyz\n’]
f.writelines(data)
f.close()
f.open(‘tmp’, ‘r’)
f.read()  # ‘abc\npqr\nxyz\n’
print(f.read()) #\n shows a new line
```

—----------

Writing Structures to a file: ‘pickle’ module

For writing structures such as list or dictionary to a file, we can use Python module ‘pickle’. Pickling refers to the process of converting the structure to a byte stream before writing to a fil. While reading the contents of the file, a reverse process called unpickling is used to convert the byte stream back to the original structure.

```
import pickle
f = open(‘file1’, ‘wb’) #b means binary
pickle.dump([‘hello’, ‘world’], f)
pickle.dump({1: ‘one’, 2: ‘two}, f)
f.close()

f=open(‘file1’, ‘rb’)
value1 = pickle.read()
value2 = pickle.read()
f.close()
```

Output: [‘hello’, ‘worl’] {1: ‘one’, 2: ‘two’}